Credit Card Fraud Detection

Applied ML & Intelligent Decision Systems Suite (2025) · Notebook 2 of 4

Yuvan Prasath B - github.com/yuvan-prasath

0.9740	0.8686	284,807	0.17%
ROC-AUC	**PR-AUC**	**Records**	**Fraud rate**
Best model score	Precision-Recall AUC	Total transactions	Class imbalance

1. Project Overview

This notebook addresses one of the most critical real-world applications of machine learning: detecting fraudulent credit card transactions in a dataset with extreme class imbalance. The challenge is not simply building a classifier — it is building one that remains reliable when only 0.17% of transactions are fraudulent, a condition where naive models achieve high accuracy by predicting everything as legitimate.

The dataset used is the publicly available Kaggle Credit Card Fraud Detection dataset, comprising 284,807 transactions made by European cardholders over two days in September 2013. Features V1 through V28 are the result of a PCA transformation applied by the dataset creators to protect cardholder privacy. The only raw features retained are Time (seconds elapsed since the first transaction) and Amount (transaction value in euros).

2. Methodology

2.1 Data Preprocessing

The dataset arrived with no missing values, making it significantly cleaner than typical clinical datasets. The primary preprocessing steps were:

- Amount and Time were standardised using StandardScaler, since V1–V28 are already PCA-scaled.
- The dataset was split 80/20 (train/test) with stratification to preserve the fraud ratio in both splits.
- SMOTE (Synthetic Minority Over-sampling Technique) was applied exclusively to the training set, generating synthetic fraud samples until both classes were balanced. The test set was left untouched to ensure evaluation integrity.

2.2 Models Trained

Three classifiers were trained on the SMOTE-resampled training data and evaluated on the original imbalanced test set:

Model	ROC-AUC	PR-AUC
Logistic Regression	0.9721	0.7134
Random Forest	**0.9740**	**0.8686**
XGBoost	0.9733	0.8412

Random Forest achieved the best PR-AUC of 0.8686, making it the selected production model. PR-AUC was prioritised over ROC-AUC as the primary evaluation metric — in cases of extreme class imbalance, ROC-AUC can be misleadingly optimistic. PR-AUC directly measures the trade-off between catching fraud (recall) and avoiding false alarms (precision), which is the operationally relevant concern.

3. Key Findings

3.1 Class Imbalance

With only 492 fraud cases out of 284,807 transactions (0.17%), this dataset represents one of the most severe class imbalance scenarios in publicly available ML benchmarks. Without SMOTE, all three models defaulted to predicting the majority class, achieving >99% accuracy but failing to detect any fraud. After SMOTE, the training set was balanced to approximately 227,454 samples per class, enabling the models to learn meaningful fraud patterns.

3.2 Feature Insights via SHAP

SHAP (SHapley Additive exPlanations) analysis on the Random Forest model identified V14, V17, V12, and V10 as the features with the highest predictive importance for fraud detection. These PCA components, while not directly interpretable in business terms, consistently separate fraudulent from legitimate transactions across the model's decision boundaries. Importantly, Amount showed relatively low SHAP importance — fraudsters deliberately use small transaction amounts to avoid detection thresholds, a behaviour confirmed by the distribution analysis in EDA.

3.3 Why PR-AUC Matters Here

A naive classifier that predicts all transactions as legitimate achieves 99.83% accuracy on this dataset — a completely useless result. ROC-AUC of ~0.97 is impressive but still susceptible to this distortion. PR-AUC of 0.8686 is the more honest signal: it tells us that at various classification thresholds, the Random Forest maintains strong precision while recovering a large proportion of actual fraud cases. In a real deployment, this directly translates to fewer fraudulent transactions going undetected and fewer legitimate transactions incorrectly blocked.

4. Technical Stack

Category	Tools / Libraries
Language	Python 3.10+
ML Framework	Scikit-learn, XGBoost
Imbalance Handling	imbalanced-learn (SMOTE)
Explainability	SHAP (TreeExplainer)
Evaluation	ROC-AUC, PR-AUC, Precision-Recall Curve
Visualisation	Matplotlib, Seaborn
Model Persistence	joblib (.pkl export)
Dataset	Kaggle Credit Card Fraud Detection (ULB)